Complete Specification and Design for Simile

Anthony Hunt

August 22, 2026

Contents

1 High-Level Design

1.1 Overview

This document serves as a living specification and design record of the very-high-level programming language Simile. Simile is founded on the union between formal specification languages and efficiency-focused programming languages, with the belief that a language can be both highly performant and highly abstract. We focus on providing a python-like/specification-language-like programming experience with a compiler strong enough to compete with hand-optimized C code. Where possible, we aim to formalize Simile's compilation process to ensure that aggressively optimized programs remain correct by construction.

Simile takes its syntactic and semantic roots in that of Event-B, a set-theory-first language used to formalize safety-oriented software specifications. Event-B notation often leans on set comprehensions (of the form $\{ x \cdot x \in S \mid E \}$) and set operations (like $S \cup T$), with progressive specification refinement at the core of the development process. Like object-oriented programs, Event-B programs tend to operate at multiple refinement levels simultaneously, especially when proving program correctness. Each atomic action must be proved correct, then proofs may be composed to prove larger programs, and so on. With Event-B as inspiration, Simile's syntax and semantics is intended to be proof-friendly in the same way, by following discrete mathematics notation.

At the same time, Simile attempts to use the semantic information available at high-level programs to aggressively optimize the executable value. For example, $X \cap T = T$ may require evaluation of $X \cap T$ in a normal programming language, but Simile attempts to match known theorems against expressions to determine that $X \cap T = T$ is only true when $X \supseteq T$. By avoiding the difficult reasoning paths of traditional imperative languages (involving side effects or mutable objects) to perform an action, set theory semantics affords an entire class of optimizations not available to other programming languages.

By building from 50 years of programming language and formal methods literature, we hope to create a language that will spark further interest in simple-to-use, efficient-to-run languages. Following sections include the high-level design of Simile, formalized subsystems, and an implementation reference.

1.2 Scope

In scope:

- We follow set theory semantics most of the time, diverging only for a particularly useful expression or programming objective.
- Programs should remain as abstract as possible, but users may provide helpful information to the compiler to unlock more aggressive optimizations.
- Notation follows that of Event-B (mostly), including a rich set of set and relational operators.

Out of scope:

- We cannot hope to become the objectively best language in all use cases. Each language has their strength, and Simile aims to be well-rounded; ergonomic enough to pick up, and fast enough to be more usable than other specification language runtimes.
- We do not follow conventional imperative language semantics that would otherwise clash with set theory optimizations.

1.3 Background

1.3.1 Programmers should not worry about implementation details

In high level programming languages, users often reach for abstract constructs like relations (dictionaries in Python, maps in Java), lists, and sets, largely because they are standardized, concise, and expressive. For example, HashMaps can be found in almost every language's standard library, and it would be peculiar to find a language that does not have first-class support for list collections. Most of the time, programmers using these types don't think about the layout or execution of their data, just that it is held within a container and operable under several composable actions (concatenation, union, intersection, lookups, etc.). However, this abstraction under the semantics of traditional high-level imperative/OO programs can lead to wasted memory and computation time, especially when composing multiple operations together. For example, the expression $S \cap (A \cup B \cup C)$ is theoretically bounded by the size of S, but a conventional language would construct several intermediary sets for $A \cup B$, then $(A \cup B) \cup C$, consuming more memory and time than required.

Low-level languages share an adjacent problem: programmers may trust primitive data type operations to be fast, but they must also tune every piece of their code to fit the type's implementation. Formally simple and powerful operations like list concatenation force the user to consider implementation details like memory allocation and ownership for both inputs and outputs. For example, in $A := S +\!\!+ T$, does the specific implementation create a new list for A containing a shallow copy of the contents of S and T? Or perhaps A and S will reference the same object, where the data in T is moved to the pointed-to value of S. Programmers must track every byte of data for straightforward operations, causing mental overhead, distractions from program correctness, and the risk of implementation errors.

Abstract data types are semantically powerful and have been implemented by hundreds of languages across thousands of libraries. Further, they can be represented by a variety of concrete types, with wildly varying levels of efficiency and effectiveness for different use cases. For example, a set of integers can be implemented by a HashSet, a bitset, an array, a tree set, or even a trie. HashSets are advantageous for element lookups, but may not be able to compare to a dense bitset in terms of memory efficiency. An array could be effective for a set that may need to be sorted, but doesnt have direct element lookups. With a little metadata-help from the programmer, the compiler should be intelligent enough to make decisions on the concrete type and optimize expressions tailored to the concrete type while completely following the abstract type's semantics.

Motivating Example: A Visitor Information System Let's turn towards an example for specification-focused programming: a group of conference administrators create a program to track resourcing for a week's worth of workshops. They already have a list of visitors, available rooms, and planned workshops for each room. A specification would model these records via carrier sets (i.e., enumerated sets):

$$Visitor = \{Ryan, Adrian, Kevin, Charlie, Michael, ...\}$$
$$Room = \{100, 101, 201, ...\}$$
$$Workshop = \{ESOP, FASE, FoSSaCS\}$$

To make this model more useful, we express relationships between these sets. For example, every workshop must occur in a room and visitors will attend workshops. Further, we can place restrictions on each relation concisely through specification language notation. If we know workshops occur in a room, it would be safe to tighten our observation that *all* workshops must occur in one room *at a time*. So, we can explicitly refine the relationship to an injective (one-to-one) function that is total on the domain (the carrier set $Workshop$). In formal notation, we write concisely:

$$location : Workshop \rightarrowtail Room$$

We can further refine another relationship: visitors can attend workshops, but visitors can only attend one workshop *at a time* and *not all* visitors may attend a workshop. This relation is actually partial on the carrier set of $Visitor$ and a

functional (many-to-one) relation. Formally:

$$attends : Visitor \nrightarrow Workshop$$

With these relations, we can query for interesting information concisely. Let's say we want to throw a surprise dinner for all attendees in a specific room. We can compose the *attends* relation with the *location* relation to find the number of people expected in a specific room. For a specific *room*, we write:

$$card((location^{-1} \circ attends^{-1})[\{room\}]$$

A HashMap would be the first concrete data structure of choice for most programmers, but without knowledge of the refined relationships, we cannot guarantee that the relational inverse operation is guaranteed to work. We set aside HashMaps for now for the sole reason that they are limited in their representation capabilities; we cannot model a many-to-many relationship using the convential definition of the type.

If we were to build a compiler for this specification language, a naive implementation might only see the broad connection between these sets and choose to safely model them as a HashSet of tuples (which can represent many-to-many relations). In this case, we concretize the types of *location* and *attends* like so:

$$location : HashSet[(Workshop, Room)]$$
$$attends : HashSet[(Visitor, Workshop)]$$

Then, each operation from the above query can be ordered in an imperative fashion:

$locationInverse : HashSet[(Room, Workshop)] := Inverse(location)$
$attendsInverse : HashSet[(Workshop, Visitor)] := Inverse(attends)$
$visitorsInRooms : HashSet[(Room, Visitor)] := Compose(locationInverse, attendsInverse)$
$visitorsInQueryRoom : HashSet[Visitor] := Lookup(room, visitorsInRooms)$
$mealsToPrepForRoom : int := Card(visitorsInQueryRoom)$

With a library of concrete functions:

function INVERSE($r : HashSet[(X, Y)]$) ▷ Runtime: $O(n)$. Memory use: $O(n)$
  $r' : HashSet[(Y, X)] := \{\}$
  for $x, y \in r$ **do**
    $r'.add((y, x))$ ▷ $s.add$ is a primitive HashSet operation mutating s
  return r'
function COMPOSE($r_1 : HashSet[(X, Y)], r_2 : HashSet[(Y, Z)]$) ▷ Runtime: $O(n^2)$. Memory use: $O(n)$
  $r' : HashSet[(X, Z)] := \{\}$
  for $x, y_1 \in r_1$ **do**
    for $y_2, z \in r_2$ **do**
      if $y_1 = y_2$ **then**
        $r'.add((x, y))$
  return r'
function LOOKUP($searchFor : X, r : HashSet[(X, Y)]$) ▷ Runtime: $O(n)$. Memory use: $O(n)$
  $r' : HashSet[Y] := \{\}$
  for $x, y \in r$ **do**
    if $searchFor = x$ **then**
      $r'.add(y)$
  return r'
function CARD($s : HashSet$) ▷ Runtime: $O(n)$. Memory use: $O(1)$

$c : int := 0$
for $x \in s$ **do**
$c := c + 1$
return c

As evident by the poor runtime and memory complexity, this is far from the most efficient representation of the system. However, it is a direct, valid translation, fulfilling all semantic requirements. A human may never write such code, but a compiler without statically analyzing the use of abstract times, may choose the broadest representation at the cost of runtime and memory usage. With knowledge of the problem domain and usage of these library functions, human programmers can intuitively decide that the *Lookup* function, for example, only ever needs to match for one room. Further, they would likely observe that this solution could be made a magnitude faster by using HashMaps instead of a tupled HashSet.

But as programmers deciding on concrete data structures, it is our responsibility to be aware of the restrictions of the concrete types and make the most appropriate decision for the expected data. A compiler should be able to make the same decisions, consulting a longer list of alternatives for further unique cases. If we inform the compiler that HashMaps must represent many-to-one relations in addition to the refined relational types mentioned above (where *location* and *attends* are one-to-one and many-to-one respectively), we effectively offload the burden of implementation choice towards a rigid, correct system.

Compare the above solution with the optimal HashMap implementation, which allocates no extra space for intermediate sets and runs in $O(n)$:

function HashMapSolution($location : HashMap[Workshop, Room], attends : HashMap[Visitor, Workshop]$)
c := 0
for $v, w \in attends$ **do**
if $location[w] = room$ **then**
$c := c + 1$
return c

Digression: relational databases Upon reviewing this example, we see strong parallels in language and functionality to relational database engines (particularly SQL). And as a data processing-focused problem, this example could of course be modelled by a relational database. For a very large event with many rooms, visitors, and workshops, perhaps a relational database is the only way to manage the data. However, using formal notation, the programmer is not burdened with the choice of that representation; they focus on correctness of the abstract data and abstract operations. The specification language intentionally hides implementation-specific operations like joins and counts, instead opting for a purer mathematical model following traditional set theory semantics.

The execution engine should have the freedom of choosing a relational database-like storage-and-query mechanism, but it should also have the freedom to choose between aggressively memory-efficient or fully-cached implementations too. In this particular example, a relational database may not even be the best choice of representation. Imagine we could encode the unchanging carrier sets (ex. a building has a limited number of *Room*s, the set of *Workshop*s is naturally limited by the number of *Room*s) as a compact bitset. If we were to perform an intersection between these encoded sets, the execution could be many times faster than the overhead from a relational database.

By providing the compute engine with more opportunities to follow unique optimization paths, we may be able to tune the computation method to the problem at hand.

1.3.2 Programmers should focus on correctness, compiler on efficiency

One of the best traits of specification languages towards correct program development is their emphasis on proving the intent of the code. Even in industry, organizing software requirements remains a bottleneck for shipping features and products. Although software developers will no doubt need to consider algorithmic efficiency when writing programs, they

should be able to abstract away primitive operations to focus on writing readable, correct, and clear code. Then, the compiler should have the freedom to swap out expressive code with a semantically equivalent, more efficient counterpart. In this case, rigorous semantics are required to demonstrate that the source program exhibits the same key behaviours as optimized programs.

High level program libraries and frameworks are already trending towards semantically rich, abstract interfaces covering a low level hand-optimized implementation. For example, the numpy library has become essential syntactic sugar for scientific computing within python. Python further serves as the standard for machine learning tooling, with libraries like PyTorch, TensorFlow, and Triton abstracting complex GPU code behind a vendor-agnostic interface.

By simplifying the language and formalizing semantics, we can grant the compiler more power over optimizations than would otherwise be possible in conventional languages.

Motivating Example: Operation Optimization in Python Let's say we have 3 sets S, T, V that can only be represented by the concrete HashSet type. Further, take our candidate query $(S \cup T) \cap V$. Python provides a suite of abstract operators for sets, so this expression may be represented as `(S | T) & V`. But its imperative semantics hide the actual computation process:

<MINTED>

Because the expected behaviour of Python operators is to create a new set while leaving the input arguments untouched, the interpreter must create a new set for the union of S and T, then another set for the intersection with V. Further, Python is forced to evaluate from left-to-right, obeying the priority of parenthesis. So, in the likely case where $S \cup T$ are larger than V, we must waste computation and memory adding and removing values from the return set (as in Step 2). The semantics of conventional imperative languages limit the compiler's ability to shuffle expression evaluation, not least because of side effects.

If we freed the semantics of these operators so that known set theorems can replace expressions, we may choose to swap $(S \cup T) \cap V$ with a more complex but computationally faster expression $(S \cap V) \cup ((T \setminus S) \cap V)$. Further, if we prevent the construction of intermediary sets and instead compute the final result directly, we can create an implementation where the resulting set never undoes prior operations, and never consumes more memory than necessary:

<MINTED>

The constructed return set strengthens the upper bound cardinality by distributing the intersection operation (which shrinks set sizes) across the union operation (which grows set sizes). As a bonus for focusing on memory efficiency, the runtime is faster too. This second version only iterates over S and T once, while the first version iterates over S, T, and $S \cup T$ twice.

High level specifications can make the intention of computation clearer and simpler, allowing simplifications to happen early in the compilation process. But we must be careful: as demonstrated with this example, the optimized answer is not necessarily the shortest formula. We must consider the concrete type representations, the atomic actions of those concrete types, and the strengths/weaknesses of those atomic actions. A new language defining only abstract semantics allows for extensions of optimizations without any behavioural differences the user would care about; the only behaviours we promise are those defined by the semantics.

1.3.3 Partial computation is powerful

Motivating Example: Maximum Tail Sum

1.4 Syntax

1.5 Semantics

1.6 Optimization

A basic strategy to optimize set and relational expressions is:

1. Normalize the expression as a set comprehensions
2. Simplify and reorganize conjuncts of the set comprehension body

The TRS for this language primarily involves lowering collection data type expressions into pointwise boolean quantifications. Breaking down each operation into set builder notation enables a few key actions:

- Quantifications over sets ($\{ x \cdot G \mid P \}$) are naturally separated into generators (G) and (non-generating) predicates (P). For sets, at least one membership operator per top-level conjunction in G will serve as a concrete element generator in generated code. Then, top level disjunctions will select one membership operation to act as a generator, relegating all others to the predicate level. For example, if the rewrite system observes an intersection of the form $\{ x \cdot x \in S \land x \in T \}$, the set construction operation must iterate over at least one of S and T. Then, the other will act as a condition to check every iteration (becoming $\{ x \cdot x \in S \mid x \in T \}$).
- By definition of generators in quantification notation, operations in G must be statements of the form $x \in S$, where x is used in the "element" portion of the set construction. Statements like $x \notin T$ or checking a property $p(x)$ must act like conditions since they do not produce any iterable elements.
- Any boolean expression for conditions may be rewritten as a combination of $\neg, \lor$, and $\land$ expressions. Therefore, by converting all set notation down into boolean notation and then generating code based on set constructor booleans, we can accommodate any form of predicate function.

2 Specification

This section contains the complete specification and low-level design of Simile, inspired by Event-B and Python.

2.1 Lexicon

Since Simile makes extensive use of Event-B-like set notation and otherwise non-ASCII characters, this section contains a list of such characters, translations to ASCII symbols where suitable, and other useful lexing information.

2.1.1 Whitespace

Similar to Python, indentation and newline tokens are significant in Simile. In particular, nested code blocks, as in if-statements, for-loops, and procedure definitions, require the use of indentations to signify the beginning and end. Indentation/newlines may also be used within collection-type comprehensions or enumerations, similar to the style seen in JSON. Comments are single-lined and start with a pound symbol (#). Indentation is ignored within bracketed expressions[1].

[1] Except for within the 'iter' block. This is a special case handled post-lexing: ignore indentation when parenthesis/bracket/brace level > 1 and we are not within the block portion of 'iter'... VBAR NEWLINE INDENT 'block' DEDENT.

2.1.2 Identifiers

Identifiers in Simile follow the standard regular expression, using only ASCII letters, underscores, and numbers: $[a-zA-Z\_][a-zA-Z\_0-9]$.

2.1.3 Primitives

Integers and floating point numbers follow a similar style to modern programming languages. The dash character ($-$) may be used in either subtraction or as a unary negation. Superscript numbers may be used for constant exponentiation. Booleans (True/False) follow Python style, with capitalized first letters. Strings are delimited with double-quotes (") and may be multilined.

2.1.4 Operators

ASCII tokens will be provided as an alternative to unicode format where suitable. The following are symbols used for expressions:

Token	ASCII Token	Name
$\cdot$	`.`	Center Dot (Quantifier Separator) [2]
.		Dot
,	`iter_and`	Comma
‘	`iter_or`	Comma
:		Colon
;		Semicolon
|		Vertical Bar
λ	`lambda`	Lambda
(		Left Parenthesis
)		Right Parenthesis
$\langle$	`<<`	Left Bracket
	`[`	Left Bracket (Alternative)
$\rangle$	`>>`	Right Bracket
	`]`	Right Bracket (Alternative)
{		Left Brace
}		Right Brace
$[\![$	`[[`	Left Double Bracket
$]\!]$	`]]`	Right Double Bracket
$=$		Equals
$\neq$	`!=`	Not Equals
$\neg$	`not`	Not
	`!`	Not (Alternative)
$\wedge$	`and`	And
$\vee$	`or`	Or
$\Rightarrow$	`=>`	Implies
$\equiv$	`==`	Equivalent

[2] Anywhere $\cdot$ is valid, . is valid, except the parser may attempt to treat . as a record field access instead of a quantifier separator (like $a.b$), so we prefer the use of $\cdot$ instead.

Token	ASCII Token	Name
$\neq$	`!==`	Not Equivalent
$\forall$	`forall`	For All
$\exists$	`exists`	There Exists
	`+`	Plus
	`-`	Minus
	`*`	Multiplication
	`/`	Division
	`div`	Integer Division
	`mod`	Modulo
	`^`	Exponent
$<$		Less Than
$\leq$	`<=`	Less Than or Equal
$>$		Greater Than
$\geq$	`>=`	Greater Than or Equal
$\in$	`in`	In (Membership)
$\notin$	`not in`	Not In (Membership)
$\cup$	`\/`	Union
$\cap$	`/\`	Intersection
$\setminus$	`\`	Backslash (Set Minus)
$\times$	`><`	Cartesian Product
$\subset$	`<<:`	Subset
$\subseteq$	`<:`	Subset or Equal
$\supset$	`:>>`	Superset
$\supseteq$	`:>`	Superset or Equal
$\not\subset$	`!<<:`	Not Subset
$\not\subseteq$	`!<:`	Not Subset or Equal
$\not\supset$	`!:>>`	Not Superset
$\not\supseteq$	`!:>`	Not Superset or Equal
$\bigcup$	`union`	General Union
$\bigcap$	`intersection`	General Intersection
$\mapsto$	`|->`	Maplet
$\oplus$	`<+>`	Relation Overriding
$\circ$	`<>`	Composition
$\mathbin{+\!\!+}$	`++`	Concatenation
$^{-1}$	`~`	Inverse
$\lhd$	`<|`	Domain Restriction
$\not\lhd$	`<<|`	Domain Subtraction
$\rhd$	`|>`	Range Restriction
$\not\rhd$	`|>>`	Range Subtraction
$\leftrightarrow$	`<->`	Relation
$\nleftrightarrow$	`<<->`	Total Relation [3]
$\twoheadleftarrow\!\!\!\!\rightarrow$	`<->>`	Surjective Relation [4]

[3] Brave Sans Mono unicode: U+E100
[4] Brave Sans Mono unicode: U+E101

Token	ASCII Token	Name
$\twoheadleftarrow\!\!\!\!\twoheadrightarrow$	`<<->>`	Total Surjective Relation [5]
$\pfun$	`+->`	Partial Function
$\fun$	`-->`	Total Function
$\pinj$	`>+>`	Partial Injection
$\inj$	`>->`	Total Injection
$\psur$	`+->>`	Partial Surjection
$\sur$	`-->>`	Total Surjection
$\bij$	`>->>`	Bijection
$..$		Upto
$\pow$	`powerset`	Powerset
$\pow_1$	`powerset1`	Non-Empty Powerset
$\sum$	`sum`	Summation
$\prod$	`product`	Product

The following are symbols used for programming constructs:

(Unicode) Token	(ASCII) Token	Name
$:=$	`:=`	Assignment
$:\in$	`::`	Choice (Non-deterministic Membership) Assignment
$\rightarrow$	`->`	Right Arrow

2.1.5 Reserved Keywords

The following list of symbols and identifiers are reserved for programming constructs:

true	false	if	else	for
while	record	enum	procedure	is
is not	return	break	continue	from
import	skip	trait	type	iter
iter_or	iter_and			

And for built-in types:

int	float	string	bool	$\mathbb{Z}$	$\mathbb{N}$
$\mathbb{N}_1$	set	sequence	bag	relation	generic
tuple					

And built-in procedures:

min	map_min	max	map_max	choice	dom
ran	card	size	sum	cast	cast_with
print	map	heads	tails	fold	

[5] Brave Sans Mono unicode: U+E102

2.2 Syntax

We follow the standard EBNF notation for describing the language structure.

Start ::= Statements

Statements ::= { SimpleStatements | CompoundStmt}

SimpleStatements ::= AssignmentOrExpr (‘;’ SimpleStmt {‘;’ SimpleStmt} NEWLINE | NEWLINE [TraitStmt])
  | (ControlFlowStmt | ImportStmt) {‘;’ SimpleStmt} NEWLINE

SimpleStmt ::= AssignmentOrExpr
  | ControlFlowStmt
  | ImportStmt

AssignmentOrExpr ::= Expr [‘:’ TypeExpr] [(‘:=’ | ‘:∈’) Expr]

TraitStmt ::= INDENT ‘`trait`’ Expr NEWLINE {‘`trait`’ Expr NEWLINE} DEDENT

Expr ::= Quantification | Predicate

Quantification ::= ‘λ’ [FlatTupleIdentifier] ‘·’ Predicate ‘|’ Expr
  | (‘∑’ | ‘∏’ | ‘⋃’ | ‘⋂’ | ‘∃’ | ‘∀’ | ‘`max`’ | ‘`min`’) QuantificationBody
  | ‘`fold`’ Assignment ‘`:`’ QuantificationBody
  | ‘`iter`’ IterQuantificationBody

QuantificationBody ::= BranchQuantificationBody
  | Generator {‘`,`’ Generator} (‘`,`’ BranchQuantificationBody | ‘|’ Expr)

BranchQuantificationBody ::= ‘`(`’ QuantificationBody ‘`)`’ {‘‘`(`’ QuantificationBody ‘`)`’}

Generator ::= IdentList ‘∈’ Expr [‘·’ Expr]

IterQuantificationBody ::= BranchIterQuantificationBody
  | GeneratorWithAssignments {‘`,`’ GeneratorWithAssignments } (‘`,`’ BranchIterQuantificationBody | ‘`->`’ IdentList ‘|’ IterBlock)

BranchIterQuantificationBody ::= ‘`(`’ IterQuantificationBody ‘`)`’ {‘‘`(`’ IterQuantificationBody ‘`)`’ }

GeneratorWithAssignments ::= Generator ‘`:`’ Assignment { ‘`;`’ Assignment}

Assignment ::= Expr [‘`:`’ TypeExpr] (‘:=’ | ‘:∈’) Expr

IterBlock ::= SimpleStmt {‘`;`’ SimpleStmt}
  | NEWLINE INDENT Statements DEDENT

IdentList ::= IdentListItem {‘`,`’ IdentListItem}

IdentListItem ::= IDENTIFIER
  | IdentListItem '↦' IdentListItem
  | '(' IdentList ')'

Predicate ::= Implication
  | Predicate ('≡' | '≢') Implication

Implication ::= Disjunction
  | Disjunction {'⇒' Disjunction}

Disjunction ::= Conjunction {'∨' Conjunction}

Conjunction ::= Negation {'∧' Negation}

Negation ::= AtomBool
  | '¬' Negation

AtomBool ::= PairExpr [('=' | '≠' | '<' | '>' | '≤' | '≥' | '∈' | '∉' | '⊂' | '⊆' | '⊃' | '⊇' | '⊄' | '⊈' | '⊅' | '⊉') PairExpr]

PairExpr ::= RelSetExpr
  | PairExpr '↦' RelSetExpr

RelSetExpr ::= SetExpr [('↔' | '⇸↠' | '↔' | '⇸' | '↠' | '→' | '↣' | '↣' | '↠' | '⇸' | '↣') RelSetExpr]

SetExpr ::= IntervalExpr
  | IntervalExpr {'∪' IntervalExpr}
  | IntervalExpr {'×' IntervalExpr}
  | IntervalExpr {'⊕' IntervalExpr}
  | IntervalExpr {'∘' IntervalExpr}
  | IntervalExpr {'∩' IntervalExpr}
  | IntervalExpr {'⫫' IntervalExpr}
  | IntervalExpr ('⩤' | '◁') IntervalExpr {'∩' IntervalExpr} [RelSubExpr]

RelSubExpr ::= ('\' | '▷' | '⩥') IntervalExpr

IntervalExpr ::= ArithmeticExpr ['..' ArithmeticExpr]

ArithmeticExpr ::= Term
  | ArithmeticExpr ('+' | '-') Term

Term ::= Factor
  | Term ('*' | '/' | 'div' | 'mod') Factor

Factor ::= [('+' | '-')] Power

Power ::= Primary ['^' Factor | ['−']('*INTEGER*' | '*FLOAT*')]

Primary ::= Atom {AtomFollow}

AtomFollow ::= '.' IDENTIFIER
  | '(' Expr {',' Expr} ')'
  | '[' Expr {',' Expr } ']'

Atom ::= INTEGER | FLOAT | STRING | BOOLEAN | IDENTIFIER
  | Set | Sequence | Bag | Tuple
  | '(' Expr ')'

Set ::= '{' CollectionBody '}'

Bag ::= '⟦' CollectionBody '⟧'

Sequence ::= '⟨' CollectionBody '⟩'

Tuple ::= '(' ')' | '(' [NEWLINE INDENT] Expr ',' [NEWLINE [INDENT]] [EnumerationBody] ')'

CollectionBody ::= QuantificationBody
  | EnumerationBody

EnumerationBody ::= [NEWLINE INDENT] Expr {',' [NEWLINE [INDENT]] Expr} [NEWLINE DEDENT]

CompoundStmt ::= IfStmt | ForStmt | WhileStmt
  | RecordStmt | ProcedureStmt

IfStmt ::= 'if' Predicate ':' Block [ElseStmt]

ElseStmt ::= 'else :' Block
  | 'else if' Predicate ':' Block [ElseStmt]

ForStmt ::= 'for' IdentList '∈' Expr ':' Block

WhileStmt ::= 'while' Expr ':' Block

RecordStmt ::= 'record' IDENTIFIER ':' NEWLINE INDENT (
    'skip' | TypedName {(',' [NEWLINE] | NEWLINE) TypedName }
) NEWLINE DEDENT

ProcedureStmt ::= 'procedure' IDENTIFIER '(' TypedName {',' TypedName } ')' '→' Expr ':' Block

Block ::= SimpleStatements
  | NEWLINE INDENT Statements DEDENT

TypedName ::= IDENTIFIER [':' TypeExpr]

TypeExpr ::= IDENTIFIER {'.' IDENTIFIER} ['[' TypeExpr {',' TypeExpr}']']
  | '(' ')'
  | '(' TypeExpr ',' [TupleTypeExprBody] ')'

TupleTypeExprBody ::= TypeExpr {‘,’ TypeExpr}

ControlFlowStmt ::= ‘`return`’ [Expr]
| ‘`break`’
| ‘`continue`’
| ‘`skip`’

ImportStmt ::= ‘`import`’ STRING
| ‘`from`’ STRING ‘`import`’ ImportList

ImportList ::= ‘`*`’
| FlatTupleIdentifier

FlatTupleIdentifier ::= IDENTIFIER {‘,’ IDENTIFIER }
| ‘(’ IDENTIFIER {‘,’ IDENTIFIER } ‘)’

2.3 Static Analysis - Type System

Let Γ be the type environment set consisting of elements either $identifier : Type$ or $Type$. The typing rules have been organized in distinct categories for readability. The notation $Type[x, y, z]$ denotes refinements on $Type$ either through instantiating parametric (generic) types or restricting values by way of `trait` clauses.

Type refinements may be applicable to types with certain traits (ex., min/max values for ordered types, definedness of expressions with certain values, set sizes).

2.3.1 Base Environment Interactions

Includes: Mechanisms for adding/extracting variables from the type environment.

$$\frac{}{\varnothing \vdash \diamond} \quad \text{(Env } \varnothing\text{)}$$

$$\frac{\Gamma \vdash A \qquad I \notin dom(\Gamma)}{\Gamma, I : A \vdash \diamond} \quad \text{(Env } I\text{)}$$

$$\frac{\Gamma_1, I : A, \Gamma_2 \vdash \diamond}{\Gamma_1, I : A, \Gamma_2 \vdash I : A} \quad \text{(Fetch Identifier)}$$

2.3.2 Subtype Interactions

Includes: Generic subtyping rules.

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \leq A} \quad \text{(Reflexive Subtype)}$$

$$\frac{\Gamma \vdash A \leq B \qquad \Gamma \vdash B \leq C}{\Gamma \vdash A \leq C} \quad \text{(Transitive Subtype)}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash A \leq B}{\Gamma \vdash a : B} \quad \text{(Subsumption)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash Any} \quad \text{(Top Type)}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \leq Any} \quad \text{(Sub Top Type)}$$

$$\frac{\Gamma \vdash A' \leq A \quad \Gamma \vdash B \leq B'}{\Gamma \vdash A \to B \leq A' \to B'} \quad \text{(Sub Function)}$$

$$\frac{\Gamma \vdash A \leq B}{\Gamma \vdash set[A] \leq set[B]} \quad \text{(Sub Set)}$$

$$\frac{\Gamma \vdash A_1 \leq B_1 \quad \Gamma \vdash A_2 \leq B_2}{\Gamma \vdash A_1 \times A_2 \leq B_1 \times B_2} \quad \text{(Sub Product)}$$

$$\frac{\Gamma \vdash A_1 \leq B_1 ... A_n \leq B_n \quad \Gamma \vdash A_{n+1} ... A_{n+m}}{\Gamma \vdash Record[I_1 : A_1, ..., I_n : A_n, I_{n+1} : A_{n+1}, ... I_{n+m} : A_{n+m}] \leq Record[I_1 : B_1, ..., I_n : B_n]} \quad \text{(Sub Record)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash T[\text{with } x] \leq T} \quad \text{(Type Refinement)}$$

2.3.3 The Set Type

Includes: Universal type; everything is a set. The empty set is a part of every type (aside from primitives).

$$\frac{\Gamma \vdash T}{\Gamma \vdash set[T]} \quad \text{(Powerset)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash EmptySet} \quad \text{(Emptyset)}$$

$$\frac{\Gamma \vdash EmptySet, set[T]}{EmptySet \leq set[T]} \quad \text{(Emptyset Bottom)}$$

$$\frac{\Gamma \vdash e_1, ..., e_n : A}{\Gamma \vdash \{e_1, ..., e_n\} : set[A]} \quad \text{(Set Enumeration)}$$

2.3.4 Primitive Types

Includes: Basic types built in to the language, like numbers (int and float), strings, and booleans. Although these could all be represented as sets, it is useful to distinguish these common types from other collections or user-defined types.

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash bool} \quad \text{(Primitives - bool)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash int} \quad \text{(Primitives - int)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash float} \quad \text{(Primitives - float)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash str} \quad \text{(Primitives - str)}$$

$$\frac{\Gamma \vdash int}{\Gamma \vdash nat \leq int[\text{with } min = 0]} \quad \text{(Nat from int)}$$

$$\frac{\Gamma \vdash int, float}{int \leq float} \quad \text{(Int from float)}$$

2.3.5 Collection Types

Includes: Sequences, relations, enums (frozen sets), and bags (multisets); Syntactic sugar for abstract data types defined in terms of sets.

$$\frac{\Gamma \vdash T_1, ..., T_n}{\Gamma \vdash tuple[T_1, ..., T_n] \quad tuple[T_1, ..., T_n] = T_1 \times ... \times T_n} \quad \text{(Tuples)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash () : EmptyTuple} \quad \text{(Empty Tuple)}$$

$$\frac{\Gamma \vdash EmptyTuple, T}{\Gamma \vdash EmptyTuple \leq tuple[T]} \quad \text{(Empty Tuple Bottom)}$$

$$\frac{\Gamma \vdash e_1 : A_1, ..., e_n : A_n}{\Gamma \vdash (e_1, ..., e_n) : tuple[A_1, ..., A_n]} \quad \text{(Tuple Enumeration)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \leftrightarrow T_2 \quad T_1 \leftrightarrow T_2 = set[tuple[T_1, T_2]]} \quad \text{(Relation Type)}$$

$$\frac{\Gamma \vdash T, nat}{\Gamma \vdash bag[T] \leq T \nrightarrow nat} \quad \text{(Bag Type)}$$

$$\frac{\Gamma \vdash e_1, ..., e_n : A}{\Gamma \vdash [\![e_1, ..., e_n]\!] : bag[A]} \quad \text{(Bag Enumeration)}$$

$$\frac{\Gamma \vdash T, nat}{\Gamma \vdash sequence[T] \leq nat \nrightarrow T} \quad \text{(Sequence Type)}$$

$$\frac{\Gamma \vdash e_1, ..., e_n : A}{\Gamma \vdash \langle e_1, ..., e_n \rangle : sequence[A]} \quad \text{(Sequence Enumeration)}$$

$$\frac{\Gamma \vdash s_1, ..., s_n : str \qquad A = \{s_1, ..., s_n\} \qquad immutable(A)}{\Gamma \vdash enum[A] \leq set[A, \text{with } immutable]} \quad \text{(Enum from static set)}$$

2.3.6 Relation Subtypes

Includes: Relation subtypes examine four key properties of relations: totality (t), surjectivity (t_r), one-to-manyness ($1-$), and many-to-oneness ($1-_r$). Syntactic sugar to produce refined relations with these properties follows the definitions presented in Event-B. See Section 5.1.3 for more details.

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \nleftrightarrow T_2 \quad T_1 \nleftrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t]} \quad \text{(Relation Subtype - Total Relation)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \nrightarrow T_2 \quad T_1 \nrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t_r]} \quad \text{(Relation Subtype - Surjective Relation)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \nrightarrow T_2 \quad T_1 \nrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t, \text{ with } t_r]} \quad \text{(Relation Subtype - Total Surjective Relation)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \nrightarrow T_2 \quad T_1 \nrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } 1-]} \quad \text{(Relation Subtype - Partial Function)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \to T_2 \quad T_1 \to T_2 = (T_1 \leftrightarrow T_2)[\text{with } t,\ \text{with } 1-]} \quad \text{(Relation Subtype - Total Function)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrowtail T_2 \quad T_1 \rightarrowtail T_2 = (T_1 \leftrightarrow T_2)[\text{with } 1-,\ \text{with } 1-_r]} \quad \text{(Relation Subtype - Partial Injection)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrowtail T_2 \quad T_1 \rightarrowtail T_2 = (T_1 \leftrightarrow T_2)[\text{with } t,\ \text{with } 1-,\ \text{with } 1-_r]} \quad \text{(Relation Subtype - Total Injection)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \twoheadrightarrow T_2 \quad T_1 \twoheadrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t_r,\ \text{with } 1-]} \quad \text{(Relation Subtype - Partial Surjection)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \twoheadrightarrow T_2 \quad T_1 \twoheadrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t,\ \text{with } t_r,\ \text{with } 1-]} \quad \text{(Relation Subtype - Total Surjection)}$$

$$\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrowtail\!\!\!\!\!\twoheadrightarrow T_2 \quad T_1 \rightarrowtail\!\!\!\!\!\twoheadrightarrow T_2 = (T_1 \leftrightarrow T_2)[\text{with } t,\ \text{with } t_r,\ \text{with } 1-,\ \text{with } 1-_r]} \quad \text{(Relation Subtype - Bijection)}$$

2.3.7 Simple Commands

Includes: Assignment, type assignment, type refinements, simple programming language commands/statements.

$$\frac{\Gamma \vdash E : A \qquad I \notin dom(\Gamma) \vee \Gamma \vdash I : A}{\Gamma \vdash I : A := E} \quad \text{(Variable Assignment)}$$

$$\frac{\Gamma \vdash T \qquad I \notin dom(\Gamma)}{\Gamma \vdash I := T} \quad \text{(Type Alias Assignment)}$$

$$\frac{\Gamma \vdash I := T}{\Gamma, I \vdash \diamond \quad I = T} \quad \text{(Type Alias)}$$

$$\frac{\Gamma \vdash E : A, E_1 : A_1, ..., E_n : A_n \qquad (I \notin dom(\Gamma) \vee \Gamma \vdash I : A)}{\Gamma \vdash I : A[\text{with } E_1 : A_1, ...,\ \text{with } E_n : A_n] := E} \quad \text{(Refined Variable Assignment)}$$

$$\frac{\Gamma \vdash T, E_1 : A_1, ..., E_n : A_n \qquad I \notin dom(\Gamma)}{\Gamma \vdash I := T[\text{with } E_1 : A_1, ...,\ \text{with } E_n : A_n]} \quad \text{(Refined Type Alias)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{break} : C} \quad \text{(Command - break)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{continue} : C} \quad \text{(Command - continue)}$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{skip} : C} \quad \text{(Command - skip)}$$

$$\frac{\Gamma \vdash E : A}{\Gamma \vdash (\text{return } E) : C} \quad \text{(Command - return)}$$

2.3.8 Quantification Expressions

Includes: Set, sequence, relation, and bag comprehensions, in addition to (set based) lambda expressions. Comprehension variables may be bound through either an $Identifier$ or an arbitrarily nested tuple of $Identifiers$

$$\frac{\Gamma, I_1 : T_1, ..., I_n : T_n \vdash E : A, P : bool \quad I_i \notin dom(\Gamma) \quad i \neq j \implies I_i \neq I_j}{\Gamma \vdash (\lambda I_1, ..., I_n \cdot P \mid E) : (T_1, ..., T_n) \nrightarrow A} \text{(Lambda Expression)}$$

$$\frac{\Gamma \vdash G : Generator[I] \quad \Gamma, I \vdash E : A}{\Gamma \vdash (G \mid E) : Quantifier[A]} \text{(Quantifier)}$$

$$\frac{\Gamma \vdash Q_i : Quantifier[A]}{\Gamma \vdash (Q_1`...`Q_n) : Quantifier[A]} \text{(Branching Quantifier)}$$

$$\frac{\Gamma \vdash G_i : Generator[T_i]}{\Gamma \vdash G_1, ...G_n : Generator[(T_1, ..., T_n)]} \text{(Generator nest)}$$

$$\frac{\Gamma \vdash S : set[T], P : bool \quad I = x_1 : T_1, ..., x_n : T_n \quad \Gamma \vdash structuralMatch(I, T)}{\Gamma \vdash (I \in S \cdot P) : Generator[T]} \text{(Generator)}$$

$$\frac{\Gamma \vdash T \quad x \notin dom(\Gamma)}{\Gamma \vdash structuralMatch(x, T)} \text{(Structural Match)}$$

$$\frac{\Gamma \vdash structuralMatch(x_1, T_1), ..., structuralMatch(x_n, T_n)}{\Gamma \vdash structuralMatch((x_1, ..., x_n), (T_1, ..., T_n))} \text{(Structural Match with Tuple)}$$

$$\frac{\Gamma \vdash Q : Quantifier[A]}{\Gamma \vdash \{\, Q \,\} : set[A]} \text{(Set Comprehension)}$$

$$\frac{\Gamma \vdash Q : Quantifier[A]}{\Gamma \vdash [\![\, Q \,]\!] : bag[A]} \text{(Bag Comprehension)}$$

$$\frac{\Gamma \vdash Q : Quantifier[A]}{\Gamma \vdash [\, Q \,] : sequence[A]} \text{(Sequence Comprehension)}$$

$$\frac{\Gamma, i : A \vdash Q : Quantifier[A] \quad \Gamma \vdash Id : A \quad i \notin dom(\Gamma)}{\Gamma \vdash (fold\ i := Id : Q) : A} \text{(Fold)}$$

$$\frac{\Gamma \vdash G : IterGenerator[I], x_i : T_i, \text{block } C}{\Gamma \vdash (G \to (x_1, ..., x_n) \mid C) : IterQuantifier[(T_1, ..., T_n)]} \text{(Iter Quantifier)}$$

$$\frac{\Gamma \vdash G : Generator[I], y_i : T_i \quad x_i \notin dom(\Gamma)}{\Gamma \vdash (G : x_1 := y_1; ...; x_n := y_n) : IterGenerator[I]} \text{(Iter Generator)}$$

$$\frac{\Gamma \vdash Q : IterQuantifier[A]}{\Gamma \vdash (iter\ Q) : A} \text{(Iter)}$$

Let $\otimes = \{\bigcup, \bigcap\}$

$$\frac{\Gamma \vdash Q : Quantifier[A]}{\Gamma \vdash (\otimes Q) : A} \text{(General Quantifiers)}$$

Let $\otimes = \{\forall, \exists\}$

$$\frac{\Gamma \vdash Q : Quantifier[bool]}{\Gamma \vdash (\otimes Q) : bool} \quad \text{(Bool Quantifiers)}$$

Let $\otimes = \{\sum, \prod\}$

$$\frac{\Gamma \vdash Q : Quantifier[T] \qquad \Gamma \vdash T \in \{int, nat, float\}}{\Gamma \vdash (\otimes Q) : T} \quad \text{(Numeric Quantifiers)}$$

2.3.9 Boolean Operations

Includes: Common predicate logic operations (equivalence, implication, and, or, etc.).
Let $\otimes = \{\equiv, \not\equiv, \implies, \wedge, \vee\}$

$$\frac{\Gamma \vdash b_1, b_2 : bool}{\Gamma \vdash b_1 \otimes b_2 : bool} \quad \text{(Binary Boolean Operations)}$$

$$\frac{\Gamma \vdash b_1 : bool}{\Gamma \vdash \neg b_1 : bool} \quad \text{(Boolean Operations - Negation)}$$

2.3.10 Equality and Membership

Includes: Equality and ordering comparisons, set membership.
Let $\otimes = \{=, \neq\}$

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2}{\Gamma \vdash a_1 \otimes a_2 : bool} \quad \text{(Equals)}$$

Let $\otimes = \{<, \leq, >, \geq\}$

$$\frac{\Gamma \vdash a_1, a_2 : T \qquad T \in \{int, float, nat\}}{\Gamma \vdash a_1 \otimes a_2 : bool} \quad \text{(Ordering Operators)}$$

Let $\otimes = \{\in, \notin\}$

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : set[T_2]}{\Gamma \vdash a_1 \otimes a_2 : bool} \quad \text{(Set Membership)}$$

2.3.11 Set Operations

Includes: Subset comparisons, union, intersection, cartesian product, etc. Also includes Maplet and Range.
Let $\otimes = \{\subset, \subseteq, \supset, \supseteq, \not\subset, \not\subseteq, \not\supset, \not\supseteq\}$

$$\frac{\Gamma \vdash a_1 : set[T_1], a_2 : set[T_2]}{\Gamma \vdash a_1 \otimes a_2 : bool} \quad \text{(Set Ordering Operations)}$$

Let $\otimes = \{\cup, \cap, \setminus\}$

$$\frac{\Gamma \vdash a_1, a_2 : set[T]}{\Gamma \vdash a_1 \otimes a_2 : set[T]} \quad \text{(Set Operations)}$$

$$\frac{\Gamma \vdash a_1 : set[T_1], a_2 : set[T_2]}{\Gamma \vdash a_1 \times a_2 : T_1 \leftrightarrow T_2} \quad \text{(Cartesian Product)}$$

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2}{\Gamma \vdash a_1 \mapsto a_2 : tuple[T_1, T_2]} \quad \text{(Maplet)}$$

$$\frac{\Gamma \vdash a_1, a_2 : int}{\Gamma \vdash a_1 \mathinner{..} a_2 : set[int, \text{ with } min = a_1, \text{ with } max = a_2]} \quad \text{(Numerical Range)}$$

$$\frac{\Gamma \vdash a : T}{\Gamma \vdash \mathbb{P}(a) : set[T]} \quad \text{(Set Operations - Powerset)}$$

2.3.12 Bag Operations

Includes: Bag union, bag intersection, etc.

Let $\otimes = \{\cup, \cap, +, -\}$

$$\frac{\Gamma \vdash a_1, a_2 : bag[T]}{\Gamma \vdash a_1 \otimes a_2 : bag[T]} \quad \text{(Bag Operations)}$$

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : bag[T_1]}{\Gamma \vdash a[b] : bag[T_2] \quad a[b] = a^{-1} \circ b} \quad \text{(Bag Operations - Image)}$$

2.3.13 Relation Operations

Includes: Relation overriding, composition, domain/range manipulation. See Section 5.1.3 for changes in relational subtypes after applying operations.

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : T_1}{\Gamma \vdash a(b) : T_2} \quad \text{(Relation Operations - Function Call)}$$

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : set[T_1]}{\Gamma \vdash a[b] : set[T_2]} \quad \text{(Relation Operations - Image)}$$

$$\frac{\Gamma \vdash a_1, a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \oplus a_2 : T_1 \leftrightarrow T_2} \quad \text{(Relation Operations - Overriding)}$$

$$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : T_2 \leftrightarrow T_3}{\Gamma \vdash a_1 \circ a_2 : T_1 \leftrightarrow T_3} \quad \text{(Relation Operations - Composition)}$$

$$\frac{\Gamma \vdash a_1 : set[T_1], a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \lhd a_2 : T_1 \leftrightarrow T_2} \quad \text{(Relation Operations - Domain Restriction)}$$

$$\frac{\Gamma \vdash a_1 : set[T_1], a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \lhd a_2 : T_1 \leftrightarrow T_2} \quad \text{(Relation Operations - Domain Subtraction)}$$

$$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : set[T_2]}{\Gamma \vdash a_1 \rhd a_2 : T_1 \leftrightarrow T_2} \quad \text{(Relation Operations - Range Restriction)}$$

$$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : set[T_2]}{\Gamma \vdash a_1 \rhd a_2 : T_1 \leftrightarrow T_2} \quad \text{(Relation Operations - Range Subtraction)}$$

2.3.14 Relational Subtypes After Operations

Includes: Changes in relational subtyping (totality, surjectivity, etc.) after applying operations. For simplicity, we implement subtype changes in a few binary operators as a boolean mask representing: [with total (t), with total on range (t_r), with one-to-manyness ($1-$), with many-to-oneness ($1-_r$)]. Subtyping is done conservatively: we keep subtype properties only when they are guaranteed. See Section 5.1.3 for more on relational subtyping.

$$\frac{\Gamma \vdash s : set[T_1], r : (T_1 \leftrightarrow T_2)[True, x, y, z]}{\Gamma \vdash (s \lhd r) : (T_1 \leftrightarrow T_2)[False, x, y, z]} \quad \text{(Relational Subtype - Domain Restriction)}$$

$$\frac{\Gamma \vdash s : set[T_1], r : (T_1 \leftrightarrow T_2)[True, x, y, z]}{\Gamma \vdash (s \lhd r) : (T_1 \leftrightarrow T_2)[False, x, y, z]} \quad \text{(Relational Subtype - Domain Subtraction)}$$

$$\frac{\Gamma \vdash s : set[T_2], r : (T_1 \leftrightarrow T_2)[x, True, y, z]}{\Gamma \vdash (r \rhd s) : (T_1 \leftrightarrow T_2)[x, False, y, z]} \quad \text{(Relational Subtype - Range Restriction)}$$

$$\frac{\Gamma \vdash s : set[T_2], r : (T_1 \leftrightarrow T_2)[x, True, y, z]}{\Gamma \vdash (r \rhd s) : (T_1 \leftrightarrow T_2)[x, False, y, z]} \quad \text{(Relational Subtype - Range Subtraction)}$$

$$\frac{\Gamma \vdash r : (T_1 \leftrightarrow T_2)[w, x, y, z]}{\Gamma \vdash r^{-1} : (T_1 \leftrightarrow T_2)[x, w, z, y]} \quad \text{(Relational Subtype - Inverse)}$$

$$\frac{\Gamma \vdash r : (T_1 \leftrightarrow T_2)[a, b, c, d], t : (T_1 \leftrightarrow T_2)[w, x, y, z]}{\Gamma \vdash (r \oplus t) : (T_1 \leftrightarrow T_2)[w, x, c \wedge y, d \wedge z]} \quad \text{(Relational Subtype - Overriding)}$$

$$\frac{\Gamma \vdash r : (T_1 \leftrightarrow T_2)[a, b, c, d], t : (T_2 \leftrightarrow T_3)[w, x, y, z]}{\Gamma \vdash (r \circ t) : (T_1 \leftrightarrow T_3)[a, b \wedge x, c \wedge y, d \wedge z]} \quad \text{(Relational Subtype - Composition)}$$

2.3.15 Sequence Operations

Includes: Concatenation

$$\frac{\Gamma \vdash a_1, a_2 : sequence[T]}{\Gamma \vdash a_1 +\!\!+ a_2 : sequence[T]} \quad \text{(Sequence Operations - Concatenation)}$$

2.3.16 Numerical Operations

Includes: Addition, subtraction, multiplication, etc. on natrual numbers, integers, and floats.

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \qquad T_i \in \{int, nat\}}{\Gamma \vdash a_1 \text{ div } a_2 : max(T_1, T_2)} \quad \text{(Integer Operations - Division)}$$

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat\}}{\Gamma \vdash a_1 \text{ mod } a_2 : max(T_1, T_2)}$$ (Integer Operations - Modulo)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 + a_2 : max(T_1, T_2)}$$ (Numerical Operations - Addition)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 - a_2 : max(T_1, T_2, int)}$$ (Numerical Operations - Subtraction)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 * a_2 : float}$$ (Numerical Operations - Floating Division)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 \text{ div } a_2 : max(T_1, T_2)}$$ (Numerical Operations - Multiplication)

$$\frac{\Gamma \vdash a_1 : T \quad T \in \{int, nat, float\}}{\Gamma \vdash -a_1 : max(int, T)}$$ (Numerical Operations - Negation)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 \hat{} \ a_2 : max(T_1, T_2)}$$ (Numerical Operations - Exponentiation)

2.3.17 Record Types

Includes: Types for named tuples.

$$\frac{\Gamma \vdash a : Record[I : A, ...]}{\Gamma \vdash a.I : A}$$ (Records - Access)

$$\frac{\Gamma \vdash A_1, ..., A_n \quad \forall I_i, I_j \cdot I_i \neq I_j}{\Gamma \vdash Record[I_1 : A_1, ..., I_n : A_n]}$$ (Records - Type Definition)

$$\frac{\Gamma \vdash a_1 : A_1, ..., a_n : A_n}{\Gamma \vdash record(a_1 = A_1, ..., a_n = A_n) : Record[a_1 : A_1, ..., a_n : A_n]}$$ (Records - Initialization)

2.3.18 Compound Commands

Includes: If, While, For, and Import statements, along with Procedure definitions and calls.

$$\frac{\Gamma \vdash C_1, C_2}{\Gamma \vdash C_1 \text{ \textbackslash n } C_2}$$ (Command - Composition)

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{import } M : C}$$ (Command - Valid Import Module)

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{from } M \text{ import } I_1, ..., I_n : C}$$ (Command - Valid Import Names)

$$\frac{\Gamma \vdash \text{import } M \text{ (with identifiers and types } I_1 : A_1, ..., I_n : A_n)}{\Gamma, M = record(I_1 = A_1, ..., I_n = A_n) \vdash \diamond}$$ (Command - Import Module)

$$\frac{\Gamma \vdash \text{from } M \text{ import } I_1, ..., I_n \text{ (with types } A_1, ..., A_n)}{\Gamma, I_1 : A_1, ..., I_n : A_n \vdash \diamond}$$ (Command - Import Names)

$$\frac{\Gamma, I : procedure[A_1, ..., A_n, R], x_1 : A_1, ..., x_n : A_n \vdash C : R \quad I, x_1, ..., x_n \notin dom(\Gamma) \quad i \neq j \implies x_i \neq x_j}{\Gamma \vdash \text{ procedure } I = C : procedure[A_1, ..., A_n, R]} \text{ (Command - Procedure Definition)}$$

$$\frac{\Gamma \vdash procedure[A_1, ..., A_n, R], a_1 : A_1, ..., a_n : A_n}{\Gamma \vdash I(a_1, ..., a_n) : R} \text{ (Command - Procedure Call)}$$

$$\frac{\Gamma \vdash b : bool, \text{block } C}{\Gamma \vdash \text{if } b : C} \text{ (Command - If)}$$

$$\frac{\Gamma \vdash b : bool, \text{block } C_1, \text{block } C_2}{\Gamma \vdash \text{if } b : C_1 \text{ else} : C_2} \text{ (Command - If Else)}$$

$$\frac{\Gamma \vdash b : bool, \text{block } C}{\Gamma \vdash \text{while } b : C} \text{ (Command - While)}$$

$$\frac{\Gamma \vdash G : set[T] \quad \Gamma, i : T \vdash C \quad i \notin dom(\Gamma)}{\Gamma \vdash \text{for } i \in G : C} \text{ (Command - For)}$$

$$\frac{\Gamma \vdash D \therefore \Gamma' \quad \Gamma' \vdash C}{\Gamma \vdash \text{block } D \text{ \textbackslash n } C} \text{ (Command - Block)}$$

2.3.19 Built-in Functions

Includes: Common set-related functions.

$$\frac{\Gamma \vdash T[\text{with } order \in traits(T)]}{\Gamma \vdash min : set[T] \to T} \text{ (Built-in - Minimum)}$$

$$\frac{\Gamma \vdash T, int}{\Gamma \vdash min : set[T] \times (T \to int) \to T} \text{ (Built-in - Mapped Minimum)}$$

$$\frac{\Gamma \vdash T[\text{with } order \in traits(T)]}{\Gamma \vdash max : set[T] \to T} \text{ (Built-in - Maximum)}$$

$$\frac{\Gamma \vdash T, int}{\Gamma \vdash max : set[T] \times (T \to int) \to T} \text{ (Built-in - Mapped Maximum)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash choice : set[T] \to T} \text{ (Built-in - Choice)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash dom : T_1 \leftrightarrow T_2 \to set[T_1]} \text{ (Built-in - Domain)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash ran : T_1 \leftrightarrow T_2 \to set[T_2]} \text{ (Built-in - Range)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash card : set[T] \to nat} \text{ (Built-in - Cardinality)}$$

$$\frac{\Gamma \vdash T}{\Gamma \vdash size : bag[T] \to nat} \text{ (Built-in - Bag Size)}$$

$$\frac{\Gamma \vdash T \quad T \in \{int, float, nat\}}{\Gamma \vdash sum : set[T] \to T} \text{ (Built-in - Sum)}$$

$$\frac{\Gamma \vdash T_1, T_2}{\Gamma \vdash cast : type[T_1] \times T_1 \to T_2} \quad \text{(Built-in - Cast)}$$

$$\frac{\Gamma \vdash T_1, T_2, x : E}{\Gamma \vdash cast : type[T_1] \times T_1 \times set[E] \to T_2[\text{with } x : E]} \quad \text{(Built-in - Cast With } ^6\text{)}$$

2.4 Static Analysis - Type System Extensions with Traits

2.4.1 Trait Collection

Traits are used to augment types, providing the compiler with additional context for the use of an abstract type that could influence the generated implementation. The following paragraphs list identified traits and provide justification for their existence.

Constant Pre-computation The type may contain values for easily identifiable constants, allowing for specialized generated instructions. When possible, constant traits will propagate along operations until the constant values can no longer be guaranteed.

- *Literal*: A trait containing a single literal value. If a type has this trait, it is guaranteed to be of this value.
- *Domain*: A looser form of *Literal* that carries a set of possible values. When applied, the type is restricted to (one of) the values from this *Domain*.
- Leave out for now: *UnknownElementInDomain*: Loosens the *Domain* guarantee so that values in the domain are only likely to appear, but not guaranteed? We would need to see if this opens up any optimizations...
- *Immutable*: Represents types whos values (and value contents) cannot be modified. Should we instead assume immutability and have a Mutable trait?

Arithmetic Restricts the value of primitive types. Collection types wishing to take advantage of this information should instead store it in the underlying primitive element type. For example, applying Min to a $set[int]$ should be written as $set[int[\text{with } min = 0]]$, not $set[int,\text{with } min = 0]$ (although we can loosen this with syntactic sugar). Like Event-B, *set* types are really powersets of the underlying element type, whereas primitive types represent values chosen from a *set* of primitive values.

- *Orderable*: Values of this type can be ordered. This may be useful for order-based implementations like BTrees for sets.
- Min: minimum value
- Max: maximum value

Collection Traits

- *Iterable*: All collections are iterable (they can be used as a for-loop generator).
- *Empty*: A collection with guaranteed cardinality = 0.
- *Size*: An over-estimate of a set's size. When available, Size will never under-estimate the number of elements in a collection.

- *UniqueElements*: All entries are guaranteed to be unique. More useful for the domain/range of relations, checking if a bag only has range = 1, or if a sequence has no repeats.
- *Total*: The elements in a set cover the entirety of the domain of the underlying element type. On relations, this means every possible pair combination is an element.

Relations Represents the relational subtype properties (injectiveness, surjectiveness, etc.)

- *TotalOnDomain*: All elements of the domain are within the mapping.
- *TotalOnRange*: All elements of the range are within the mapping.
- *ManyToOne*: Elements of the domain may be repeated with different range values.
- *OneToMany*: Elements of the range may be repeated with different range values. *ManyToOne* and *OneToMany* imply a one-to-one mapping

Generics

- *GenericBound*: Contains the set of types a generic may become. For example, T[with $GenericBound = \{int, float\}$ represents any numeric value.

2.4.2 Trait-trait Interactions

Collection of trait-implies-trait interactions that are universal (i.e., they do not depend on a type). Most traits are really just boolean values, as in, does this type have this property. Thus when traits used with boolean operations should be interpreted as truthy/falsy.

Interpret these as if it were type initialization rules. So, if a type were initialized with these traits, we would get the following implications "for-free". A type can only have one of each trait (except for multiple literals?), so traits given in the assumption can be used in the result without ambiguity or naming. Traits with function notation (parenthesis) are assumed to be new constructions.

TODO design decision: should traits be applied automatically or just verified? If we apply automatically, we could give best effort and then error/remove traits that are incompatible. In verification, we just throw an error. We may use trait restrictions per-type to further determine incompatibilities (ex. an integer cannot be empty?).

$$\frac{Domain = \{\}}{\neg Domain} \quad \text{(Domain Empty)}$$

$$\frac{Literal \quad \neg Domain}{Domain := \{Literal\}} \quad \text{(Literal implies a Domain)}$$

$$\frac{Literal \quad Domain}{Domain := Domain \cup Literal} \quad \text{(Literal within Domain)}$$

$$\frac{Domain \quad Orderable \quad \neg Min}{Min := min(Domain)} \quad \text{(Orderable Domain without Min)}$$

$$\frac{Domain \quad Orderable \quad Min}{Min := min(Min, Domain)} \quad \text{(Orderable Domain with Min)}$$

$$\frac{Domain \quad Orderable \quad \neg Max}{Max = max(Domain)} \quad \text{(Orderable Domain without Max)}$$

$$\frac{Domain \quad Orderable \quad Max}{Max = max(Max, Domain)}$$ (Orderable Domain with Max)

$$\frac{Min}{Orderable}$$ (Min implies Order)

$$\frac{Max}{Orderable}$$ (Max implies Order)

$$\frac{UniqueElements \quad Size = len(Domain) \quad \neg Empty}{Total}$$ (Full set)

$$\frac{Size = 0}{Empty}$$ (Empty Size)

$$\frac{Size \neq 0}{\neg Empty}$$ (Non-Empty Size)

$$\frac{Size}{Iterable}$$ (Size implies Iterable)

$$\frac{Literal \quad Orderable \quad \neg Min}{Min = Literal}$$ (Orderable Literal is Min)

$$\frac{Literal \quad Orderable \quad \neg Max}{Max = Literal}$$ (Orderable Literal is Max)

2.4.3 Type-trait interactions

Some types are incompatible with some traits, and some types must have some traits. This section lists the possible traits for each type - asterisks are for mandatory traits, and traits not listed are not allowed for that specific type.

All types Literal and Domain support constant propagations and compile time computations.

- Immutable
- Literals
- Domain

Bool

- Domain = {true, false}*

None

- Literal*
- Domain? (implied from literal)

String

- Empty
- Size (length)
- Unique?

- Iterable*
- Orderable (lexicographic)*

Int, Float

- Min
- Max
- Size (num of bits? or just depend on min/max? If so, min/max must be given, with a default value of 64 bit range)
- Orderable*

Tuple

- Min
- Max
- Size (length)
- UniqueElements
- Total
- Iterable*
- Empty
- Orderable (lexicographic if dependents are orderable?)

Maplet Tuple but with the addition of:

- Domain?
- Size = 2*

Record

- Iterable* (treat as a relation)
- Domain* (str entries)
- ManyToOne* (treat as a relation with keys = strings)

Procedure

- Domain should be stored in the domain of the input types since there are too many to count? Range should be the domain of the return type
- Need to worry about generics?

Set

- Orderable if children are
- Min
- Max
- Iterable*
- Empty
- UniqueElements*
- Size
- Total

Relation All traits except generic trait

Bag Same as relation, with ManyToOne*. Should we add a trait called Count, which is the sum of the range values?

Seq Same as relation, with ManyToOne*, Size*

Enum

1. Immutable*
2. Domain*
3. Literal*
4. Iterable*
5. UniqueELements*
6. Size*?
7. Can never be total unless enum size = 1, then always total

2.4.4 Operation-Trait Interactions

Operations on *Literal* traits will always produce a *Literal* where possible, following the effects of applying the function on values. For conciseness, we omit all *Literal* interactions and instead refer to the semantics of the operations themselves.

$$\frac{a : T[Max = x] \quad b : T[Min = y] \quad x < y}{(a < b) : Bool[Literal = true]} \quad \text{(Less than with max-min)}$$

$$\frac{a : T[Max = x] \quad b : T[Min = y] \quad x \leq y}{(a \leq b) : Bool[Literal = true]} \quad \text{(Less than or equal with max-min)}$$

$$\frac{a : T[Min = x] \quad b : T[Max = y] \quad x > y}{(a < b) : Bool[Literal = false]}$$ (Less than with min-max)

$$\frac{a : T[Min = x] \quad b : T[Max = y] \quad x > y}{(a \leq b) : Bool[Literal = false]}$$ (Less than or equal with min-max)

$$\frac{a : Int[Literal] \quad b : Int[Literal]}{(a \,..\, b) : set[Int[Min = a, Max = b], Literal = \{a, a+1, ..., b\}, Total]}$$ (Upto)

$$\frac{s : set[Empty]}{(s = \{\}) : Bool[Literal = true]}$$ (Is Empty)

$$\frac{s : set[T, Size = x] \quad a : T}{(s \cup \{a\}) : set[T, Size = x + 1]}$$ (Set add)

$$\frac{s : set[T, Size = x, Total] \quad a : T}{(s \cup \{a\}) : set[T, Size = x, Total]}$$ (Set add with Total)

$$\frac{s : set[T, Size = x] \quad a : T}{(s \setminus \{a\}) : set[T, Size = x]}$$ (Set delete)

$$\frac{s : set[T, Size = x, Total] \quad a : T}{(s \setminus \{a\}) : set[T, Size = x - 1]}$$ (Set delete removes Total)

$$\frac{s : set[T[Domain = S]] \quad a : T[Literal = A]}{(a \in s) : Bool[Literal = A \in S]}$$ (Membership within domain)

$$\frac{s : set[T, Total] \quad a : T}{(a \in s) : Bool[Literal = true]}$$ (Membership within total domain)

$$\frac{a_1, a_2, ..., a_n : T}{\{a_1, a_2, ..., a_n\} : set[T, Size = n]}$$ (Enumeration)

$$\frac{s : set[T, Size = n]}{card(s) : Int[Max = n]}$$ (Cardinality)

$$\frac{s : set[T, Size = n, Total]}{powerset(s) : set[set[T], Size = 2^n, Total]}$$ (Powerset)

$$\frac{s : set[T, Empty]}{choice(s) : TypeError}$$ (Empty choice fails (statically))

$$\frac{s : set[T[Max = y, Size = n]]}{sum(s) : T[Max = y \times n]}$$ (Sum bounds)

$$\frac{s : set[T[Max = y, Size = n]]}{sum(s) : T[Max = y^n]}$$ (Product bounds)

$$\frac{s : set[T[Min = y]] \quad a : T[Literal = y] \quad a \in s}{min(s) : T[Literal = y]}$$ (Min looks for Min)

$$\frac{s : set[T[Max = y]] \quad a : T[Literal = y] \quad a \in s}{max(s) : T[Literal = y]}$$ (Max looks for Max)

$$\frac{s : set[T, Literal = a] \quad t : set[T, Literal = b]}{(s \cup t) : set[T, Literal = a \cup b]} \text{ (Union Literal)}$$

$$\frac{s : set[T[Domain = a]] \quad t : set[T[Domain = b]]}{(s \cup t) : set[T[Domain = a \cup b]]} \text{ (Union with widest Domain)}$$

$$\frac{s : set[T[Min = a]] \quad t : set[T[Min = b]] \quad a < b}{(s \cup t) : set[T[Min = a]]} \text{ (Union with lowest Min)}$$

$$\frac{s : set[T[Max = a]] \quad t : set[T[Max = b]] \quad a < b}{(s \cup t) : set[T[Max = b]]} \text{ (Union with highest Max)}$$

$$\frac{s : set[T, Empty] \quad t : set[T, x] \quad a < b}{(s \cup t) : set[T, x]} \text{ (Union with Empty keeps Traits)}$$

$$\frac{s : set[T, Size = x] \quad t : set[T, Size = y] \quad x < y}{(s \cup t) : set[T, Size = y]} \text{ (Union with widest size)}$$

$$\frac{s : set[T] \quad t : set[T, Total]}{(s \cup t) : set[T, Total]} \text{ (Union with Total stays)}$$

$$\frac{s : set[T[GenericBound = x]] \quad t : set[T[GenericBound = y]]}{(s \oplus t) : set[T[GenericBound = x \cap y]]} \text{ (Generics with set ops)}$$

$$\frac{s : set[T[Domain = x]] \quad t : set[V[Domain = y]]}{(s \times t) : set[tuple[T, V, Domain = x \times y]]} \text{ (Cartesian Product)}$$

$$\frac{s : set[T, Total] \quad t : set[V, Total]}{(s \times t) : set[tuple[T, V, Total]]} \text{ (Total Cartesian Product)}$$

$$\frac{s : set[T, Min = w, Max = x] \quad t : set[T, Min = y, Max = z] \quad w > y \quad x < z}{(s \subset t : set[T, Min = w, Max = x]) : Bool} \text{ (Subset with min-max)}$$

$$\frac{r : relation[T[Domain = x], V[Domain = y]]}{r^{-1} : relation[V[Domain = y], T[Domain = x]]} \text{ (Inverse swaps Domain)}$$

$$\frac{s : relation[T, V, Empty] \quad t : relation[V, U]}{s \circ t : relation[T, U, Empty]} \text{ (Empty Composition)}$$

$$\frac{r : relation[T, V] \quad s : set[T, Empty]}{r[s] : set[V, Empty]} \text{ (Image with Empty Arg)}$$

$$\frac{r : relation[T, V, Empty] \quad s : set[T]}{r[s] : set[V, Empty]} \text{ (Image with Empty Relation)}$$

$$\frac{r : relation[T, V, TotalOnDomain] \quad s : set[T, Domain = domain(r), Size > 0]}{r[s] : set[V, Size > 0]} \text{ (Image with ManyToOne)}$$

$$\frac{r : relation[T, V, ManyToOne] \quad s : set[T, Size = n]}{r[s] : set[V, Size = n]} \text{ (Image with ManyToOne)}$$

$$\frac{r : relation[T, V, TotalOnDomain]}{domain(r) : set[T, Total]} \text{ (Total domain)}$$

$$\frac{r : relation[T, V, TotalOnRange]}{range(r) : set[V, Total]} \quad \text{(Total range)}$$

$$\frac{s : sequence[T, Size = x] \qquad t : sequence[T, Size = y]}{s +\!\!+ t : sequence[T, Size = x + y]} \quad \text{(Concat with Size)}$$

2.5 Definedness

2.6 Language Examples

3 Features and Behaviours

4 (Abstract) Data Types

4.1 Traits

4.2 Boolean

4.3 Integer

4.4 Float

4.5 String

4.6 Set

4.7 Relation

4.8 Sequence

4.9 Bag

4.10 Tuple

4.11 Record

4.12 Procedure

4.13 Generic

4.13.1 Operations

5 User-Facing Specification

5.1 (Abstract) Data Types

For the most part, Simile's semantics resemble those of set theory and specification languages (specifically, Event-B). Aside from a few primitive types that abstract over bare set theory and programming language specific structures, like integer arithmetic and procedure definitions, all objects inherit properties from sets. For example, a sequence can be modelled as a set of pairs from natural numbers to the sequence element type.

In the forthcoming sections, we explicitly state the design of these set-theory-inspired types, useful properties, and underlying implementations.

5.1.1 Primitives

With consideration to execution efficiency and the extensive amount of existing optimization in modern languages, Simile makes use of the following basic types:

Booleans As the cornerstone of logical reasoning, booleans deserve an explicit type with reserved identifiers: $\mathbb{B} = \{True, False\}$. Further, Simile provides a healthy supply of common operations beyond $\wedge$ and $\vee$: $\implies, \equiv, \impliedby, \forall, \exists$, etc.

Integers While integers are a very mature type in the world of computing, trade-offs between arbitrary-precision and fixed-point arithmetic may require testing and careful review.

If Simile chooses to represent numbers through conventional fixed point arithmetic, struggles with overflow and extremely large numbers are compounded by the innate non-determinism within Simile. For example, calculating the sum of a set of very large numbers can be done in arbitrary order, according to the semantics of mathematics (and Simile). But in fixed-point arithmetic, the resulting value can very well differ based on the order of operations. In other words, commutative math operations are no longer commutative under fixed precision. At some point in the computation pipeline, Simile will need to decide the order in which to evaluate the set, which may cause hard-to-spot bugs and unexpected runtime failures.

Conversely, choosing arbitrary precision integers may bring Simile operators closer to mathematics at the cost of speed. The core features of Simile, while targeting formal methods users, attempt to efficiently execute set and relation operations. Adding checks for overflow on every mathematical operation may prove detrimental to performance.

The implementation decision essentially is one of context vs. safety. To benchmark Simile with these types, we gather a list of examples and judge whether the performance hit is worth the reliability benefits. Specific questions to examine: How often does overflow occur in these examples? Do numbers approach the upper limits of fixed precision often enough to warrant accommodating infrastructure? What is the performance of fixed vs. arbitrary precision calculations.

Floating Point Numbers In a similar vein to integers, floats are an imperfect translation of real numbers into efficiently calculable representations. Since floats (or even reals) are not used as often in specifications as integers, we leave the implementation as the conventional 64-bit representation. However, we note that non-determinism on these operations will prove especially tricky, since the result of commutative complements may differ without explicit over/underflow.

Strings Strings are syntactic sugar over Sequences of characters, represented internally as a sequence of (fixed precision) integers. Eventually, Simile aims to support built-in string operations as seen in many common programming languages.

5.1.2 Sets

Foundational Simile type: an unordered, unique collection of objects.

Sets may be used in two ways:

1. Enumerations by way of $\{x, y, z\}$ (read: The set of elements x, y, and z)

2. Comprehensions of the form $\{ x \cdot x \in S \mid f(x) \}$ (read: The set of function f applied to all elements in S). A small effort is made to accommodate a shorthand form, wherein $\{ x \cdot x \in S \mid x \}$ may be written as $\{ x \cdot x \in S \}$, but this syntax easily confuses bound/unbound variables and is not recommended for complex comprehension expressions.

Sets, as mathematical objects with only two major restrictions, lend themselves well to a variety of computational implementations. Arrays, ordered arrays, hash maps, bit sets, roaring bit sets, path maps, and tries can all be used to satisfy the uniqueness property and hide an internally ordered representation from users.

No matter the underlying implementation, the optimizer may make use of syntactically-analyzed set sizes to direct the choice of lowered iterator. Further, recording the cardinality of a set at runtime may prove beneficial to the performance of common cardinality-based specification expressions at a very small memory cost. Other properties based on element type, like max/min for numeric types, limits for numeric ranges, maximum sizes, may be useful.

5.1.3 Relations

Relations are sets that make use of paired/maplet elements (i.e., they define a relationship between two sets). Like sets, implementations may vary greatly, with the potential to increase efficiency of relation-specific operations.

Like sets, relations may benefit from compile-time computed properties: size of the domain, size of the range, totality, many-to-oneness, $O(1)$ access to the domain and range, and access to associated domain/range properties.

Relational Subtypes The notion of a relation is a broad classification of a set of pairs, where the domain, range, and relationship are not bound by any conditions. However, when generating code based on purely mathematical relations, it can be advantageous to use notions of totality, injectivity, surjectivity, etc. In particular, these named relational subtypes can be described with four properties: total on the domain, total on the range, one-to-many, and many-to-one.

The below table converts conventional notation into these four subtypes:

Relational Subtype	Properties			
	T	TR	1-	1-R
Relation				
				✓
Partial Function			✓	
Partial Injection			✓	✓
Surjective Relation		✓		
		✓		✓
Partial Surjection		✓	✓	
		✓	✓	✓
Total Relation	✓			
	✓			✓
Total Function	✓		✓	
Total Injection	✓		✓	✓
Total Surjective Relation	✓	✓		
	✓	✓		✓
Total Surjection	✓	✓	✓	
Bijection	✓	✓	✓	✓

Table 3: Conventional mathematical notation of relational subtypes decomposed into four properties: domain totality (T), range totality (TR), domain one-to-manyness (1-), and range one-to-manyness (1-R)

Subtype Properties Properties that can affect code generation may involve:

- New properties after applying operations on sets/relations
- Size of result
- Validity of operations
- Deterministic nature of operations
- Super/subset properties
- Relational identities

Below is a list of the impact of these properties.
Domain Totality:

- $R[S] \neq \varnothing$
- $R(S)$ is always valid
- R^{-1} is TR
- $|R| \geq dom(R)$

Range Totality:

- R^{-1} is T
- $|R| \geq ran(R)$

Domain One-to-manyness:

- $R(\{e\})$ is deterministic
- $|R[\{e\}]| = 1$
- $|R[S]| \leq |S|$
- $|R| \leq dom(R)$

Range One-to-manyness:

- $|R| \leq ran(R)$

5.1.4 Bags

Bags (or Multisets) are a refinement of relations wherein, for a bag with element type T, the bag is a relational function from $T \to \mathbb{Z}^+$, denoting the count of items in a collection. Once the count of an item reaches 0, the item is removed from the bag.

Bags must be many-to-one and carry specialized behaviours for conventional relation operations. For example, relational image ($R[B]$) with a bag argument will now return a bag, where the number of occurrences of the resulting item replaces what would otherwise be a regular set of those items.

Additional fields to record the size of the bag (sum of the range) may prove useful.

5.1.5 Sequences

Sequences are yet another refinement of relations where a sequence with element type T represents a relational function from $\mathbb{N} \to T$ that is total on some defined bounds.

The dense nature of sequence domains, along with knowledge of sequence sizes, may open up further data type refinements.

5.2 Data Type Operations

5.2.1 Primitives

5.2.2 Sets

5.2.3 Relations

5.2.4 Bags

5.2.5 Sequences

6 Implementation Specification

6.1 Abstract Data Types

6.2 Atomic Operations

6.3 Optimization Frontend

6.4 Optimization Backends

7 AST Lowering and Optimization Rules

Below is a list of rewrite rules for key abstract data types, syntactic sugar, and some builtin functions. Phases are intended to be executed in order; the post-condition of one phase serves as the pre-condition for the next.

7.1 Syntactic Sugar for Bags

$$R[B] \rightsquigarrow R^{-1} \circ B \qquad \text{(Bag Image)}$$

$$S \cup T \rightsquigarrow [\![x \cdot x \in dom(S) \cup dom(T) \wedge n = max(S[x] \cup T[x]) \mid x \mapsto n]\!] \qquad \text{(Bag Predicates - Union)}$$

$$S \cap T \rightsquigarrow [\![x \cdot x \in dom(S) \cap dom(T) \wedge n = min(S[x] \cup T[x]) \wedge n > 0 \mid x \mapsto n]\!] \qquad \text{(Bag Predicates - Intersection)}$$

$$S + T \rightsquigarrow [\![x \cdot x \in dom(S) \cup dom(T) \wedge n = sum(S[x] \cup T[x]) \mid x \mapsto n]\!] \qquad \text{(Bag Predicates - Sum)}$$

$$S - T \rightsquigarrow [\![x \cdot x \in dom(S) \wedge n = S(x) - (T \cup [\![x \mapsto 0]\!])(x) \wedge n > 0 \mid x \mapsto n]\!] \qquad \text{(Bag Predicates - Difference)}$$

7.2 Syntactic Sugar for Sequences

$$L +\!\!+ M \rightsquigarrow L \cup \{ x \mapsto y \cdot x \mapsto y \in M \mid x + max(dom(L)) \mapsto y \} \qquad \text{(Concatenation)}$$

$$flatten(X) \rightsquigarrow foldL(+\!\!+, [], X) \qquad \text{(Flatten)}$$

$$foldL(f, e, X) \rightsquigarrow foldL(f, f(e, first(X)), tail(X)) \qquad \text{(Fold Left Associative)}$$

$$foldL(f, e, []) \rightsquigarrow e \quad \text{(Fold Left Associative - Empty)}$$
$$first(X) \rightsquigarrow X(0) \quad \text{(First)}$$
$$tail(X) \rightsquigarrow \{\, i \mapsto x \cdot i \mapsto x \in X \wedge i \neq 0 \mid i - 1 \mapsto x \,\} \quad \text{(Tail)}$$
$$append(X, y) \rightsquigarrow X \cup \{max(dom(X)) \mapsto y\} \quad \text{(Append)}$$
$$prepend(X, y) \rightsquigarrow \{\, i \mapsto x \cdot i \mapsto x \in X \mid i + 1 \mapsto x \,\} \cup 0 \mapsto y \quad \text{(Prepend)}$$

7.3 Builtin Functions

$$f(x) := E \rightsquigarrow f := f \oplus \{x \mapsto E\} \quad \text{(Functional Override)}$$
$$card(S) \rightsquigarrow \sum x \cdot x \in S \mid 1 \quad \text{(Cardinality)}$$
$$dom(R) \rightsquigarrow \{\, x \mapsto y \cdot x \mapsto y \in R \mid x \,\} \quad \text{(Domain)}$$
$$ran(R) \rightsquigarrow \{\, x \mapsto y \cdot x \mapsto y \in R \mid y \,\} \quad \text{(Range)}$$
$$R \oplus Q \rightsquigarrow Q \cup (dom(Q) \lhd R) \quad \text{(Override)}$$
$$S \lhd R \rightsquigarrow \{\, x \mapsto y \cdot x \mapsto y \in R \wedge x \in S \mid x \mapsto y \,\} \quad \text{(Domain Restriction)}$$
$$S \lhd\!\!\!- R \rightsquigarrow \{\, x \mapsto y \cdot x \mapsto y \in R \wedge x \notin S \mid x \mapsto y \,\} \quad \text{(Domain Subtraction)}$$
$$R \rhd S \rightsquigarrow \{\, x \mapsto y \cdot x \mapsto y \in R \wedge y \in S \mid x \mapsto y \,\} \quad \text{(Range Restriction)}$$
$$R \rhd\!\!\!- S \rightsquigarrow \{\, x \mapsto y \cdot x \mapsto y \in R \wedge y \notin S \mid x \mapsto y \,\} \quad \text{(Range Subtraction)}$$
$$size(S) \rightsquigarrow \sum x \mapsto n \in S \cdot n \quad \text{(Bag Size)}$$

7.4 Comprehension Construction

Intuition All set-like variables and literals are decomposed into set comprehensions.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.

$$R(x) \rightsquigarrow choice(R[x]) \quad \text{(Functional Image [a])}$$
$$R[S] \rightsquigarrow \{\, x \mapsto y \cdot x \mapsto y \in R \wedge x \in S \mid y \,\} \quad \text{(Image)}$$
$$x \mapsto y \in S \times T \rightsquigarrow x \in S \wedge y \in T \quad \text{(Product)}$$
$$x \mapsto y \in R^{-1} \rightsquigarrow y \mapsto x \in R \quad \text{(Inverse)}$$
$$x \mapsto y \in (Q \circ R) \rightsquigarrow x \mapsto z \in Q \wedge z' \mapsto y \in R \wedge z = z' \quad \text{(Composition)}$$

[a] May fail if R is not total. *choice* is nondeterministic.

$$S \cup T \rightsquigarrow \{\, x \cdot x \in S \vee x \in T \,\} \qquad \text{(Predicate Operations - Union)}$$
$$S \cap T \rightsquigarrow \{\, x \cdot x \in S \wedge x \in T \,\} \qquad \text{(Predicate Operations - Intersection)}$$
$$S \setminus T \rightsquigarrow \{\, x \cdot x \in S \wedge x \notin T \,\} \qquad \text{(Predicate Operations - Difference)}$$
$$x \in \oplus(E \mid P) \rightsquigarrow P \wedge x = E \qquad \text{(Membership Collapse [a])}$$

[a] Rule only matches inside the predicate of a quantifier. Explicitly enumerating all matches for all quantuantification types and predicate cases (ANDs, ORs, etc.) would require too much boilerplate. x must be bound by the encasing quantifier.
The $\oplus$ operator represents any quantifier that returns a set-like type (ex. generalized union/intersection, set comprehension, relation comprehension).

7.5 Disjunctive Normal Form

Intuition All quantifier predicates are expanded to DNF (i.e. $\wedge$-operations nested within top-level $\vee$-operations).

Notes All matches of this phase occur only inside quantifier predicates.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Quantifier predicates are in disjunctive normal form - top level or-clauses with inner and-clauses.
- If no $\vee$ operators exist within a quantifier predicate, the predicate must only contain $\wedge$ operators.

$$x_1 \wedge ... \wedge (x_i \wedge x_{i+1}) \wedge ... \rightsquigarrow x_1 \wedge ... \wedge x_i \wedge x_{i+1} \wedge ... \qquad \text{(Flatten Nested } \wedge\text{)}$$
$$x_1 \vee ... \vee (x_i \vee x_{i+1}) \vee ... \rightsquigarrow x_1 \vee ... \vee x_i \vee x_{i+1} \vee ... \qquad \text{(Flatten Nested } \vee\text{)}$$
$$\neg\neg x \rightsquigarrow x \qquad \text{(Double Negation)}$$
$$\neg(x \vee y) \rightsquigarrow \neg x \wedge \neg y \qquad \text{(Distribute De Morgan - Or)}$$
$$\neg(x \wedge y) \rightsquigarrow \neg x \vee \neg y \qquad \text{(Distribute De Morgan - And)}$$
$$x \wedge (y \vee z) \rightsquigarrow (x \wedge y) \vee (x \wedge z) \qquad \text{(Distribute } \wedge \text{ over } \vee\text{)}$$

7.6 Or-wrapping

Intuition Restructuring quantifier predicates with top-level ORs for later rules.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Quantifier predicates are in disjunctive normal form - top level or-clauses with inner and-clauses.

$$\{\, E \cdot \bigwedge P_i \,\} \rightsquigarrow \{\, E \cdot \bigvee \bigwedge P_i \,\} \qquad \text{(Or-wrapping [a])}$$

[a] To simplify the matching process later on, we wrap every top-level AND statement (which is guaranteed to be a ListOp by the dataclass field type definition) with an OR.

7.7 Generator Selection

Intuition All nested ANDs are placed in a tree structure to better suit loop lowering.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Each top-level or-clause within quantification predicates must have one selected 'generator' predicate (GeneratorSelectionPredicate - GSP) of the form $x \in S$ that loops over its bound dummy variable x.
- All conjunctive clauses are wrapped in either a GSP or CombGSP structure.
- Quantifier predicates are in disjunctive normal form (with GSP replacing AND structures) - top level or-clauses with inner and-clauses.

Notes All matches of this phase occur only inside quantifier predicates. GSP is a tuple of form (generator chain, predicates) that can be flattened to a conjunctive clause. A CombGSP is a triple of form (shared generator, disjunctive children predicates/generators, predicates) that can likewise be flattened into a conjunctive clause. We keep the structure of these tuples distinct to ease the rewriting process.

Brainstorming selection heuristics Conditions to consider:

- Prefer composition chains in case of nested loops (ex. $x \mapsto y \in R \wedge y \mapsto z \in Q$, but what about renaming? - $x \mapsto y \in R \wedge y' \mapsto z \in Q \wedge y = y'$). Perhaps this could just be a preference for relations over sets if we see a nested quantifier.
- Choose most common generator among a list of or-clauses (allows for greater simplification later on)
- Choose smallest generator (by set size). This information may not always be accessible (ex. if a set is created by reading values from standard input). Functions may need to choose this on a case-by-case basis (ie. one function call could have two args, with a small set on the left arg and a large set on the right arg. But what if the sizes are reversed later in the code? We've already statically lowered it).
- When should constant folding happen? Equality substitution necessary for this?
- This may need to work with nesting considerations.
- What if we try moving these optimizations to *loop* structures far later in the pipeline?

$$\bigwedge P_i \rightsquigarrow GSP(\bigwedge P_{g_i}, \bigwedge_{P_i \notin P_g} P_i) \qquad \text{(GSP Wrapping [a])}$$

$$GSP(x \wedge xs, P) \vee GSP(x \wedge ys, Q) \rightsquigarrow CombGSP(x, GSP(xs, P) \vee GSP(ys, Q)) \qquad \text{(Nested Generator Selection [b])}$$

[a] GSP is a tuple-like structure, where all entries can be flattened into an AND. The LH term must occur inside a quantifier's predicate - one generator per top-level or-clause. Chained generators may be necessary to allow for nested loop operations (ie. non-top-level clauses may require generators). P_g is a list of chained generators, specific set membership clauses (of form $x \in S$) distinguished from the rest of $\bigwedge P_i$. Currently, the selection of P_g is arbitrary (and thus the rewrite system is not confluent), but heurestics may be added later to choose better generators.

[b] $free$ is a function that returns true when variables in the predicate are defined (either bound by the current generator or bound prior to the current statement)

7.8 GSP to Loop

Intuition Start lowering expressions into imperative-like loops.

Post-condition

- All quantifiers are transformed into *loop* structures.
- *loop* predicates are of the form $x \in S \wedge \bigwedge P_i$.
- All *loop* predicates have an assigned generator of the form $x \in S$.

$$\oplus E \mid P \rightsquigarrow \begin{array}{l} a := \mathit{identity}(\oplus) \\ \textbf{loop } P: \\ \quad a := \mathit{accumulate}(a, E) \end{array} \qquad \text{(Quantifier Generation [a])}$$

[a] $\oplus$ works for any quantifier (but not $\forall$ and $\exists$). The identity and accumulate functions are determined by the realized $\oplus$. For example, if $\oplus = \sum$, the identity is 0 and accumulate is addition.

$$\begin{array}{l} \textbf{loop } \bigvee P_i: \\ \quad \text{body} \end{array} \rightsquigarrow \begin{array}{l} \textbf{loop } P_0 \\ \quad \mathit{body} \\ \textbf{loop } P_1 \wedge \neg P_0 \\ \quad \mathit{body} \\ \textbf{loop } P_2 \wedge \neg \bigvee_{i<2} P_i \\ \quad \mathit{body} \end{array}$$

(Top-level Or-Loop)

$$\begin{array}{l} \textbf{loop } \mathit{GSP}(g \wedge gs, P) \\ \quad \text{body} \end{array} \rightsquigarrow \begin{array}{l} \textbf{loop } \mathit{SingleGSP}(g, \mathit{free}(P)) \\ \quad \textbf{loop } \mathit{GSP}(gs, \mathit{bound}(P)) \\ \quad\quad \text{body} \end{array}$$

(Chained GSP Loop [a])

[a] free considers defined variables at this point in time.

$$\begin{array}{l} \textbf{loop } \mathit{GSP}([], P) \\ \quad \text{body} \end{array} \rightsquigarrow \begin{array}{l} \textbf{if } P \textbf{ then} \\ \quad \text{body} \end{array}$$

(Empty GSP Loop)

$$\begin{array}{l} \textbf{loop } \mathit{CombGSP}(g, gs, P) \\ \quad \text{body} \end{array} \rightsquigarrow \begin{array}{l} \textbf{loop } \mathit{SingleGSP}(g, P) \\ \quad \textbf{loop } gs_0 \\ \quad\quad \text{body} \\ \quad \textbf{loop } gs_1 \\ \quad\quad \text{body} \\ \quad \ldots \end{array}$$

(Combined GSP Loop)

7.9 Relational Subtyping Loop Simplification

Intuition Simplify unnecessary iteration structures into direct accesses.

Post-condition

- All quantifiers are transformed into *loop* structures.
- *loop* predicates are of the form $x \in S \wedge \bigwedge P_i$.
- All *loop* predicates have an assigned generator of the form $x \in S$.

No change from previous, but the number of loop structures should not increase.

$$x \in dom(R) \rightsquigarrow True \qquad \text{(Total membership elimination [a])}$$
$$x \in ran(R) \rightsquigarrow True \qquad \text{(Surjective membership elimination [b])}$$

[a] R is total and x satisfies the type of R's domain. Currently unused since it may eliminate some generators - move higher in the list?
[b] R is surjective and x satisfies the type of R's range. Currently unused since it may eliminate some generators - move higher in the list? Or

loop $SingleGSP(x \mapsto y \in R, x = a \wedge P)$
    body

$\rightsquigarrow$

if $a \in dom(R)$ **then**
    loop $SingleGSP(y \in R[a], P[x := a])$
        body

(Concrete Domain Image)

loop $SingleGSP(x \mapsto y \in R, y = a \wedge P)$
    body

$\rightsquigarrow$

if $a \in ran(R)$ **then**
    loop $SingleGSP(x \in R^{-1}[a], P[y := a])$
        body

(Concrete Range Image [a])

[a] Requires efficient lookups for ran, inverse, and image

loop $SingleGSP(y \in R[x], P)$
    body

where $free(x)$ and R is many-to-one

$\rightsquigarrow$

if $P[y := R(a)]$ **then**
    $body[y := R(a)]$

(Single Element Loop)

loop $SingleGSP(x \in \{y\}, P)$
    body

$\rightsquigarrow$

if $P[x := y]$ **then**
    $body[x := y]$

(Singleton Membership Elimination)

7.10 Loop Code Generation

Intuition Eliminate all intermediate loop structures.

Post-condition

- AST is in imperative code style (for loops, if statements, etc).
- All *loop* and quantification constructs have been eliminated.
- Some variables may not be defined.

loop $SingleGSP(P_g, \bigwedge P_i)$
body

$\rightsquigarrow$

if $\bigwedge_{free(P_i)} P_i$ **then**
for P_g **do**
if $\bigwedge_{bound(P_i)} P_i$ **then**
body

(Conjunct Conditional [a])

[a] Function *free* returns clauses in P that contain only free + defined variables. *bound* returns the clauses that contain the bound variable x or undefined variables. P_g is the selected generator.

7.11 Replace and Simplify

Intuition Eliminate all undefined variables (with implicit $\exists$ quantifiers).

Post-condition

- AST is in imperative code style (for loops, if statements, etc).
- All variables are defined.

if $Identifier(x) = E \wedge P$ **then**
body

$\rightsquigarrow$

if $P[x := E]$ **then**
body[x := E]

(Equality Elimination [a])

[a] x is an undefined, unbound variable in the current scope. E is an expression that does not contain x. Simplify resulting booleans.

$$P \wedge True \rightsquigarrow P \quad \text{(Simplify And-true)}$$
$$P \wedge False \rightsquigarrow False \quad \text{(Simplify And-false)}$$
$$P \vee True \rightsquigarrow True \quad \text{(Simplify Or-true)}$$
$$P \vee False \rightsquigarrow P \quad \text{(Simplify Or-false)}$$
$$Statement(Statement(x)) \rightsquigarrow Statement(x) \quad \text{(Flatten Nested Statements)}$$

References

[1] Christophe Métayer and Laurent Voisin. "The Event-B Mathematical Language". 2007.